

Thought Process — MediQueue

Problem statement

Small clinics run queues on paper or by shouting names across the waiting room. Patients have no idea how long they'll wait, and receptionists repeat "you're next" dozens of times a day instead of focusing on actual patients. MediQueue fixes this with two synced screens: one for the receptionist to manage the queue, one for patients to see live status on their own phone — answering three questions: can a token be assigned in under 10 seconds, does the patient screen update without refreshing, and is the wait estimate based on real data instead of a guess?

Process

We started by mapping the real-time requirement first, since it was the riskiest part: Socket.IO broadcasting one shared queue:updated event to every connected screen, so receptionist and patient views never disagree. Data modeling came next — a Token schema with status (waiting/serving/served) and a singleton QueueState tracking the current token and a rolling average consultation time. The wait-time formula was the key design decision: instead of hardcoding "5 minutes per patient," we measured the actual duration of each consultation as it happened and folded it into a running average, so estimates get more accurate the longer the clinic uses it. We designed the UI in Stitch first — testing empty states, disabled buttons, and reconnect states before writing code — then built the backend with an automatic in-memory fallback so demos never break if MongoDB is unreachable.

Key process steps

1. Mapped the real-time event contract (addPatient, callNext, setAvgTime → queue:updated) before writing any UI.
2. Designed receptionist + patient screens in Stitch, including empty/disabled/offline states.
3. Built atomic queue logic (Token + QueueState models) with Jest tests before wiring up sockets.
4. Replaced the hardcoded wait-time guess with a rolling average computed from real consultation durations.
5. Deployed backend (Render) and frontend (Vercel) separately, then verified live sync across multiple tabs.

Why Socket.IO, and why one shared broadcast

Polling would technically work, but introduces a delay window and wastes requests when nothing has changed. Socket.IO pushes updates the instant a mutation happens. Critically, every mutation — whether it came from the receptionist UI or a REST call — broadcasts to **every** connected client, including the sender. This means the receptionist screen and every patient display always render from the identical state snapshot. There is no "local optimistic update that might drift" on one screen and a "server truth" on another — there is only one truth, broadcast everywhere.

How wait time is actually computed

$$\text{avgConsultMin} = ((\text{avgConsultMin} * \text{totalServed}) + \text{actualDurationMin}) / (\text{totalServed} + 1)$$

Every time `callNext` fires, the system measures how long the *previous* consultation actually took (now - `calledAt`) and folds it into this rolling average. Each waiting patient's individual estimate is then:

`estimatedWaitMin = tokensAhead * avgConsultMin`

A receptionist can still manually override the average (useful before any real data exists, e.g. a clinic's first day), which resets the counter so future real data isn't diluted by stale history.

Concurrency and edge cases

Case	Mitigation
Two rapid "Call Next" clicks	Button disabled immediately on click until the server acknowledges the update.
Race condition on token counter	Token IDs and the current-token pointer are updated through atomic database operations (<code>findOneAndUpdate</code> with <code>\$inc</code>), never a separate read-then-write.
Calling next on an empty queue	Server validates the waiting list before acting and returns a clean <code>QUEUE_EMPTY</code> error rather than corrupting state.
Duplicate patient submissions	Input + button disabled until server acknowledgment; token IDs are always assigned server-side, never trusted from the client.
Client disconnects mid-action	Socket.IO auto-reconnects; on reconnect, the client re-fetches the full snapshot once via REST to resync, covering any missed broadcast.
Multiple receptionist tabs open	Atomic writes mean whichever request lands first wins; both tabs then receive the identical broadcast afterward, staying consistent.
Server restart or DB hiccup	Queue state is persisted in MongoDB, not in-memory by default, so it survives restarts; if MongoDB is unreachable, the backend falls back to in-memory automatically so the demo still runs.

Results

Adding a patient and assigning a token takes one input field and the Enter key — well under 10 seconds. Every connected patient screen updates instantly when "Call Next" is clicked, with no manual refresh, verified across multiple simultaneous browser tabs on the live deployment. The wait-time estimate is recalculated from real elapsed consultation time after every patient, so it reflects how this specific clinic actually runs rather than a generic assumption.

What I'd do differently

I'd add basic authentication before deploying anywhere beyond a demo, since right now anyone with the link could act as the receptionist. I'd also build a small analytics view on top of the consultation-time data already being collected, since that history is currently calculated but never shown to the clinic owner. Lastly, I'd add a lightweight SMS/WhatsApp fallback for patients who'd rather not stare at a screen while waiting, rather than assuming everyone wants a live display.